Student 29 **NITHISH RAAJU V** 192411016RD 5 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.Scanner;
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StringBuffer editHistory = new StringBuffer();

        System.out.println("Enter a paragraph:");
        String paragraph = sc.nextLine();

        editHistory.append("Original Document Entered.\n");

        int charCount = paragraph.length();

        int wordCount;
        String trimmed = paragraph.trim();
        if (trimmed.isEmpty()) wordCount = 0;
        else wordCount = trimmed.split("\\s+").length;

        String[] sentencesParts = paragraph.split("[.!?]+");
        int sentenceCount = 0;
        for (String s : sentencesParts) {
            if (!s.trim().isEmpty()) sentenceCount++;
        }
        if (paragraph.trim().length() > 0 && sentenceCount == 0) sentenceCount = 1;

        editHistory.append("Counted Characters, Words, and Sentences.\n");

        System.out.println("Enter the word to replace:");
        String oldWord = sc.nextLine().trim();
        System.out.println("Enter the new word:");
        String newWord = sc.nextLine().trim();

        String replacedText = paragraph.replaceAll(
                "\\b" + Pattern.quote(oldWord) + "\\b",
                Matcher.quoteReplacement(newWord)
        );
        editHistory.append("Replaced '" + oldWord + "' with '" + newWord + "'.\n");

        String[] parts = replacedText.split("(?<=[.!?])\\s*");
        StringBuilder reversedSentences = new StringBuilder();

        for (String sentence : parts) {
            String s = sentence.trim();
            if (!s.isEmpty()) {
                StringBuilder sb = new StringBuilder(s);
                sb.reverse();
                reversedSentences.append(sb).append(" ");
            }
        }

        editHistory.append("Reversed each sentence individually.\n");

        StringBuilder finalDocument = new StringBuilder();
        finalDocument.append("---- Final Formatted Document ----\n");
        finalDocument.append(reversedSentences.toString().trim());


        editHistory.append("Final document constructed successfully.\n");
```

```
        System.out.println("\nOriginal Document:");
        System.out.println(paragraph);

        System.out.println("\nCharacter Count: " + charCount);
        System.out.println("Word Count: " + wordCount);
        System.out.println("Sentence Count: " + sentenceCount);

        System.out.println("\nText After Word Replacement:");
        System.out.println(replacedText);

        System.out.println("\nEditing History:");
        System.out.println(editHistory.toString());

        System.out.println("\n" + finalDocument.toString());

        sc.close();
    }
}
```

INPUT

```
Java is fun.Learning is powerful.practice makes perfect.
fun
awesome
```

OUTPUT

```
(no output)
```

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

public class HospitalTokenManagementSystem {
    static class Patient {
        String id;
        String name;
        String dept;
        List<String> appointmentIds = new ArrayList<>();

        Patient(String id, String name, String dept) {
            this.id = id;
            this.name = name;
            this.dept = dept;
        }
    }

    static Patient findById(List<Patient> patients, String pid) {
        for (Patient p : patients) {
            if (p.id.equalsIgnoreCase(pid)) return p;
        }
        return null;
    }

    static void removeCancelled(List<Patient> patients, String appointmentId) {
        boolean removed = false;
        for (Patient p : patients) {
            Iterator<String> it = p.appointmentIds.iterator();
            while (it.hasNext()) {
                String a = it.next();
                if (a.equalsIgnoreCase(appointmentId)) {
                    it.remove();
                    removed = true;
                    break;
                }
            }
            if (removed) break;
        }
        if (!removed) System.out.println("NOT FOUND");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n;
        try {
            n = sc.nextInt();
        } catch (Exception e) {
            return;
        }

        List<Patient> patients = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            String pid = sc.next();
            String pname = sc.next();
            String dept = sc.next();
            if (findById(patients, pid) == null) {
                patients.add(new Patient(pid, pname, dept));
            }
        }
```

```
        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {
            int choice = sc.nextInt();

            if (choice == 1) {
                String pid = sc.next();
                String pname = sc.next();
                String dept = sc.next();
                if (findById(patients, pid) == null) {
                    patients.add(new Patient(pid, pname, dept));
                }
            } else if (choice == 2) {
                for (Patient p : patients) {
                    System.out.println(p.id + " " + p.name + " " + p.dept);
                }
            } else if (choice == 3) {
                String pid = sc.next();
                String newName = sc.next();
                String newDept = sc.next();
                Patient p = findById(patients, pid);
                if (p == null) System.out.println("NOT FOUND");
                else {
                    p.name = newName;
                    p.dept = newDept;
                }
            } else if (choice == 4) {
                String pid = sc.next();
                String appointmentId = sc.next();
                sc.next();
                sc.next();
                Patient p = findById(patients, pid);
                if (p == null) System.out.println("NOT FOUND");
                else p.appointmentIds.add(appointmentId);
            } else if (choice == 5) {
                String appointmentId = sc.next();
                removeCancelled(patients, appointmentId);
            } else if (choice == 6) {
                String dept = sc.next();
                boolean found = false;
                for (Patient p : patients) {
                    if (p.dept.equalsIgnoreCase(dept)) {
                        System.out.println(p.id + " " + p.name + " " + p.dept);
                        found = true;
                    }
                }
                if (!found) System.out.println("NO PATIENTS");
            } else if (choice == 7) {
                if (patients.isEmpty()) {
                    System.out.println("NO PATIENTS");
                } else {
                    Patient first = patients.get(0);
                    Patient last = patients.get(patients.size() - 1);
                    System.out.println(first.id + " " + first.name + " " + first.dept);
                    System.out.println(last.id + " " + last.name + " " + last.dept);
                }
            }
        }

        sc.close();
    }
}
```

INPUT

```
2
p001 karthik cardiology
p002 priya neurology
3
1 p001 karthik cardiology
5 a101
2
```

OUTPUT

```
(no output)
```

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Set<Integer> active = new HashSet<>();
        Set<Integer> revoked = new HashSet<>();

        if (!sc.hasNextInt()) {
            sc.close();
            return;
        }

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            String name = sc.next();
            int id = sc.nextInt();
            String status = sc.next().trim().toUpperCase();

            if (status.equals("REVOKED")) {
                revoked.add(id);
            } else {
                active.add(id);
            }
        }

        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {
            int id = sc.nextInt();
            if (revoked.contains(id)) System.out.println("INVALID");
            else if (active.contains(id)) System.out.println("VALID");
            else System.out.println("INVALID");
        }

        int r = sc.nextInt();
        for (int i = 0; i < r; i++) {
            int id = sc.nextInt();
            if (active.contains(id)) {
                active.remove(id);
                revoked.add(id);
            }
        }

        int v = sc.nextInt();
        for (int i = 0; i < v; i++) {
            int id = sc.nextInt();
            if (revoked.contains(id)) System.out.println("INVALID");
            else if (active.contains(id)) System.out.println("VALID");
            else System.out.println("INVALID");
        }

        sc.close();
    }
}
```

INPUT

```
3
Alice 101 ACTIVE
Bob 102 REVOKED
```

```
Charlie 103 ACTIVE
3
101
102
999
1
103
2
103
101
```

OUTPUT

```
(no output)
```

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        LinkedHashSet<String> insertionOrder = new LinkedHashSet<>();
        TreeSet<String> sortedOrder = new TreeSet<>();

        while (true) {
            System.out.println("\n1. Add certificate");
            System.out.println("2. Verify certificate");
            System.out.println("3. Display insertion order");
            System.out.println("4. Display sorted order");
            System.out.println("5. Total unique certificates");
            System.out.println("6. Remove revoked certificate");
            System.out.println("0. Exit");
            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 0) break;

            if (choice == 1) {
                String id = sc.nextLine().trim();
                boolean added = insertionOrder.add(id);
                if (added) {
                    sortedOrder.add(id);
                    System.out.println("Certificate added.");
                } else {
                    System.out.println("Duplicate certificate ID. Not added.");
                }
            } else if (choice == 2) {
                String id = sc.nextLine().trim();
                if (insertionOrder.contains(id)) System.out.println("Certificate exists.");
                else System.out.println("Certificate does not exist.");
            } else if (choice == 3) {
                if (insertionOrder.isEmpty()) System.out.println("No certificates.");
                else {
                    for (String id : insertionOrder) System.out.println(id);
                }
            } else if (choice == 4) {
                if (sortedOrder.isEmpty()) System.out.println("No certificates.");
                else {
                    for (String id : sortedOrder) System.out.println(id);
                }
            } else if (choice == 5) {
                System.out.println("Total unique certificates: " + insertionOrder.size());
            } else if (choice == 6) {
                String id = sc.nextLine().trim();
                if (insertionOrder.remove(id)) {
                    sortedOrder.remove(id);
                    System.out.println("Revoked certificate removed.");
                } else {
                    System.out.println("Certificate not found.");
                }
            }
        }

        sc.close();
```

```
    }
}
```

INPUT

```
1
CERT123
1
CERT456
2
CERT123
3
4
5
6
CERT456
3
0
```

OUTPUT

```
(no output)
```

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class Main {
    private final Map<String, Product> inventory = new HashMap<>();

    static class Product {
        private final String id;
        private final String name;
        private final String category;
        private int quantity;
        private final double pricePerUnit;
        private boolean discontinued;

        public Product(String id, String name, String category, int quantity, double pricePerUnit) {
            this.id = id;
            this.name = name;
            this.category = category;
            this.quantity = quantity;
            this.pricePerUnit = pricePerUnit;
            this.discontinued = false;
        }

        public String getId() { return id; }
        public String getName() { return name; }
        public String getCategory() { return category; }
        public int getQuantity() { return quantity; }
        public double getPricePerUnit() { return pricePerUnit; }
        public boolean isDiscontinued() { return discontinued; }

        public void setQuantity(int quantity) {
            this.quantity = quantity;
        }

        public void discontinue() {
            this.discontinued = true;
        }

        public double totalValue() {
            return quantity * pricePerUnit;
        }

        @Override
        public String toString() {
            return "ID=" + id +
                    ", Name=" + name +
                    ", Category=" + category +
                    ", Quantity=" + quantity +
                    ", Price=" + pricePerUnit +
                    ", TotalValue=" + totalValue();
        }
    }

    public void addProduct(String id, String name, String category, int quantity, double pricePerUnit) {
        inventory.put(id, new Product(id, name, category, quantity, pricePerUnit));
    }

    public boolean updateQuantity(String id, int newQuantity) {
        Product p = inventory.get(id);
        if (p == null) return false;
        p.setQuantity(newQuantity);
        return true;
```

```
    }

    public Product searchProduct(String id) {
        return inventory.get(id);
    }

    public double calculateTotalInventoryValue() {
        double sum = 0.0;
        for (Product p : inventory.values()) {
            if (!p.isDiscontinued()) sum += p.totalValue();
        }
        return sum;
    }

    public Map<String, List<Product>> displayCategoryWiseProducts() {
        Map<String, List<Product>> byCategory = new HashMap<>();
        for (Product p : inventory.values()) {
            if (p.isDiscontinued()) continue;
            byCategory.computeIfAbsent(p.getCategory(), k -> new ArrayList<>()).add(p);
        }
        return byCategory;
    }

    public boolean removeDiscontinuedProduct(String id) {
        Product p = inventory.get(id);
        if (p == null) return false;
        p.discontinue();
        inventory.remove(id);
        return true;
    }

    public List<Product> displayAllAvailableProducts() {
        List<Product> list = new ArrayList<>();
        for (Product p : inventory.values()) {
            if (!p.isDiscontinued()) list.add(p);
        }
        return list;
    }

    public static void main(String[] args) {
        Main app = new Main();

        app.addProduct("P001", "Laptop", "Electronics", 15, 55000);
        app.addProduct("P002", "Smartphone", "Electronics", 30, 22000);
        app.addProduct("P003", "Headphones", "Accessories", 60, 1800);
        app.addProduct("P004", "Coffee Beans", "Grocery", 25, 850);
        app.addProduct("P005", "Keyboard", "Accessories", 40, 1200);

        app.updateQuantity("P002", 28);

        Product found = app.searchProduct("P002");
        System.out.println(found != null ? found : "Product not found");

        System.out.println("Total Inventory Value = " + app.calculateTotalInventoryValue());

        Map<String, List<Product>> categoryWise = app.displayCategoryWiseProducts();
        for (String category : categoryWise.keySet()) {
            System.out.println("\n" + category);
            for (Product p : categoryWise.get(category)) {
                System.out.println(p);
            }
        }

        app.removeDiscontinuedProduct("P004");
```

```
        System.out.println("\nAll Available Products:");
        for (Product p : app.displayAllAvailableProducts()) {
            System.out.println(p);
        }
    }
}
```

OUTPUT

```
(no output)
```

Student 30 **OSURI TEJASREE** 192371043RD 0 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

— Not Submitted —

Q2. Hospital Token Management System using List Interface

— Not Submitted —

Q3. Airport Boarding Queue Management using Queue Interface

— Not Submitted —

Q4. Digital Certificate Verification using Set Interface

— Not Submitted —

Q5. Smart Inventory Management using Map Interface

— Not Submitted —